

4.5 Project Structure Overview

4.5.1 Overview

The CampusCare project follows a modular full-stack architecture that separates the frontend mobile application from the backend API server. This structure improves maintainability, scalability, and organization.

The project is divided into two main folders:

```
campuscare-project/  
├── backend/  
└── mobile/
```

Folder	Purpose
backend	Contains the Node.js + Express backend APIs and database logic
mobile	Contains the React Native + Expo mobile application

4.5.2 Backend Structure

The backend is implemented using Node.js and Express.js. It follows a layered architecture with separate folders for routes, controllers, middleware, and configuration files.

Backend Folder Structure

```
backend/  
├── src/  
│   ├── controllers/  
│   ├── middleware/  
│   ├── routes/  
│   ├── config/  
│   └── uploads/  
├── package.json  
├── .env  
└── index.js
```

Backend Folder Descriptions

Folder/File	Purpose
controllers/	Contains the business logic for authentication, issue management, comments, and worker operations
middleware/	Contains JWT authentication and role-based authorization middleware
routes/	Defines API endpoints and connects them to controllers
config/	Stores configuration files such as Supabase connection setup
uploads/	Handles temporary uploaded image files
package.json	Contains backend dependencies and npm scripts

Folder/File	Purpose
<code>.env</code>	Stores environment variables and secret keys
<code>index.js</code>	Main backend entry point that starts the Express server

Backend Route Files

File	Purpose
<code>authRoutes.js</code>	Authentication APIs such as login and register
<code>issueRoutes.js</code>	Issue creation, retrieval, assignment, and status APIs
<code>workerRoutes.js</code>	Worker-specific issue operations
<code>managerRoutes.js</code>	Facility Manager operations
<code>commentRoutes.js</code>	Comment management APIs
<code>userRoutes.js</code>	User-related APIs

4.5.3 Frontend Structure

The frontend is implemented using React Native and Expo. The application follows a screen-based architecture using React Navigation for role-based navigation.

Frontend Folder Structure

```
mobile/
├── src/
│   ├── api/
│   ├── navigation/
│   ├── screens/
│   ├── components/
│   ├── context/
│   └── assets/
├── App.js
├── package.json
└── app.json
```

Frontend Folder Descriptions

Folder/File	Purpose
<code>api/</code>	Contains Axios API configuration and backend communication logic
<code>navigation/</code>	Contains role-based navigation structure using React Navigation
<code>screens/</code>	Contains all application screens for Community Members, Managers, and Workers
<code>components/</code>	Contains reusable UI components
<code>context/</code>	Stores authentication and global state management
<code>assets/</code>	Stores application images and static assets

Folder/File	Purpose
<code>App.js</code>	Main frontend entry point
<code>package.json</code>	Contains frontend dependencies
<code>app.json</code>	Expo application configuration

Frontend Screen Structure

The `screens` folder is organized based on user roles.

```
screens/
├── auth/
├── community/
├── manager/
└── worker/
```

Folder	Purpose
<code>auth/</code>	Login and registration screens
<code>community/</code>	Community Member screens
<code>manager/</code>	Facility Manager screens
<code>worker/</code>	Worker screens

4.5.4 Architecture Summary

The CampusCare system follows a client-server architecture.

Frontend Responsibilities

- Display mobile UI
- Handle navigation
- Capture images
- Send API requests
- Store JWT tokens locally

Backend Responsibilities

- Handle API requests
- Authenticate users
- Enforce role-based access control
- Process issue management operations
- Connect to Supabase database and storage

Database Responsibilities

- Store users, issues, comments, and photos

- Maintain relational data integrity
- Support issue lifecycle tracking

4.5.5 Design Advantages

The project structure provides several advantages:

- Separation of concerns
- Easier debugging and maintenance
- Better scalability
- Cleaner API organization
- Improved code readability
- Reusable frontend components
- Secure backend authorization structure

This modular architecture makes the CampusCare system easier to maintain and extend in future versions.

```
JS WorkerDashboard.js
JS WorkIssueScreen.js
JS ProfileScreen.js
(i) README.md
JS App.js
{} package-lock.json
{} package.json
(i) README.md
> node_modules
. .gitignore
. erdddddddddddddd.png
{} package-lock.json
{} package.json
```

EXPLORER

✓ CAMPUSCARE-PR...    

✓ mobile

✓ src

✓ navigation

✓ screens

✓ auth

JS LoginScreen.js

JS RegisterScreen.js

✓ community

JS CommunityDashboard.js

JS IssueDetailsScreen.js

JS MyIssuesScreen.js

JS SubmitIssueScreen.js

✓ manager

JS AssignIssueScreen.js

JS ManagerDashboard.js

JS UpdateStatusScreen.js

✓ worker

JS WorkerDashboard.js

JS WorkIssueScreen.js

JS ProfileScreen.js

(i) README.md

JS App.js

{} package-lock.json

{} package.json

> OUTLINE

> TIMELINE

